



University of Information Technology and Sciences

Department of Computer Science and Engineering

L A B A S S I G N M E N T

Solving Four LeetCode Problems Using C++

Course Name	Data Structures and Algorithms I Lab
Course Code	CSE0613212
Course Session	Autumn 2026

S U B M I T T E D T O

Md. Yousuf Ali
Lecturer, Course Teacher

S U B M I T T E D B Y

Kazi Rabit Jahir
Student ID: 04325205101006
Batch: 58
Section: A1

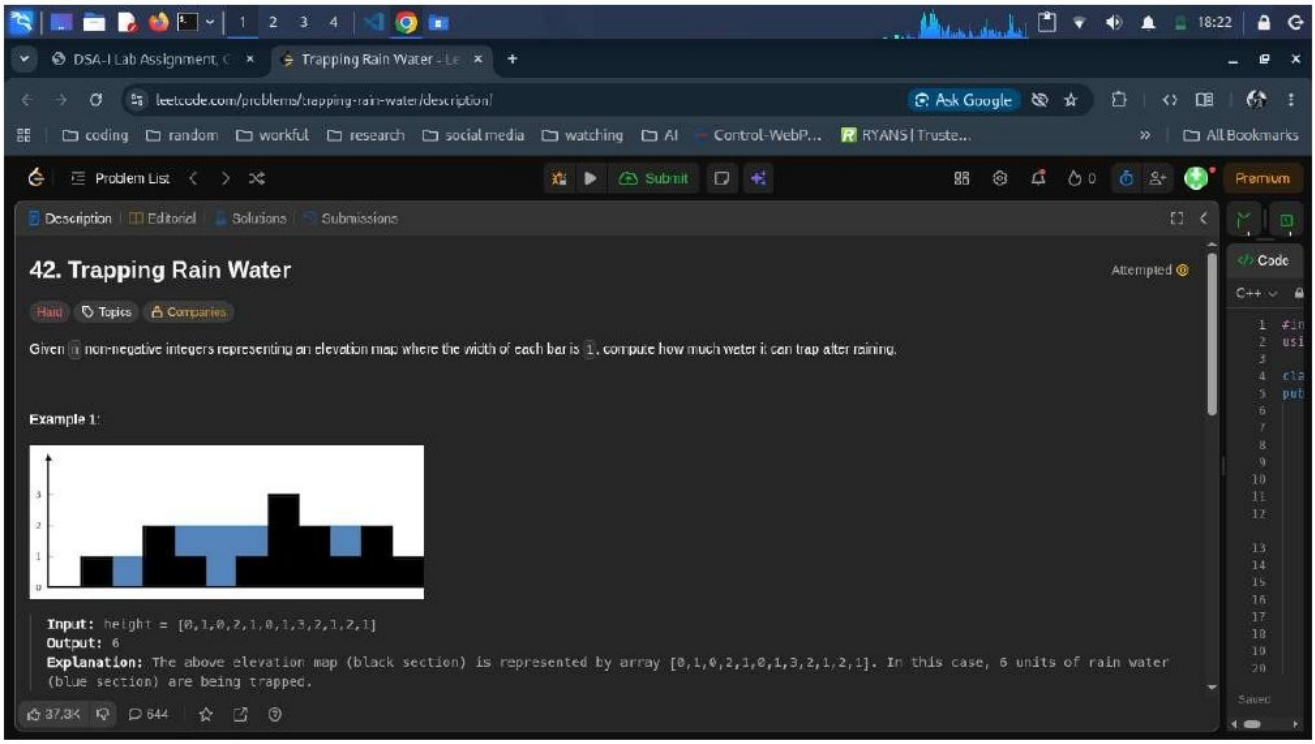
Table of Contents

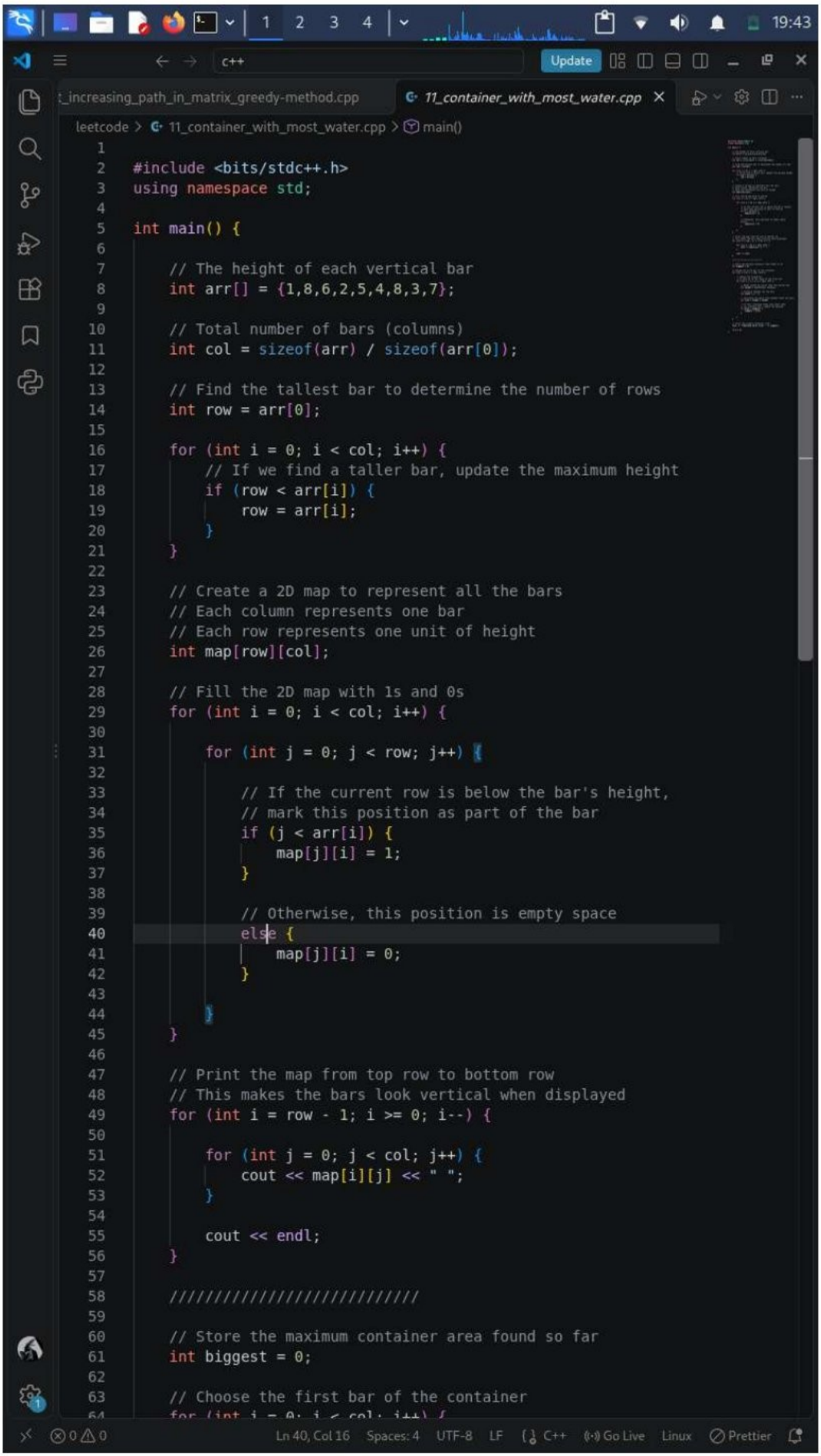
Question 1: Trapping Rain Water.....3
LeetCode 42 (Hard) – https://leetcode.com/problems/trapping-rain-water/

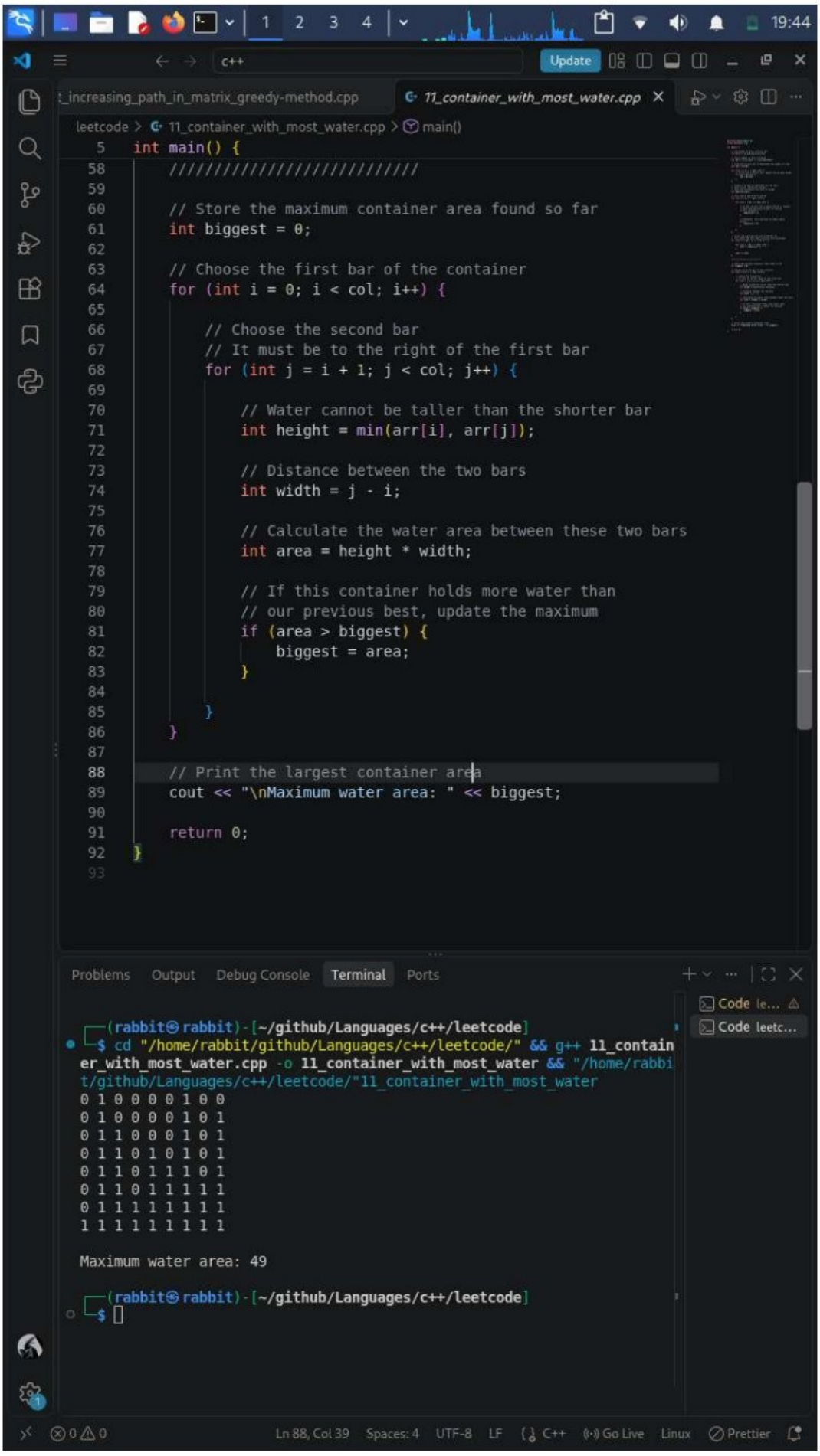
Question 2: Reverse Nodes in k-Group.....6
LeetCode 25 (Hard) – https://leetcode.com/problems/reverse-nodes-in-k-group/

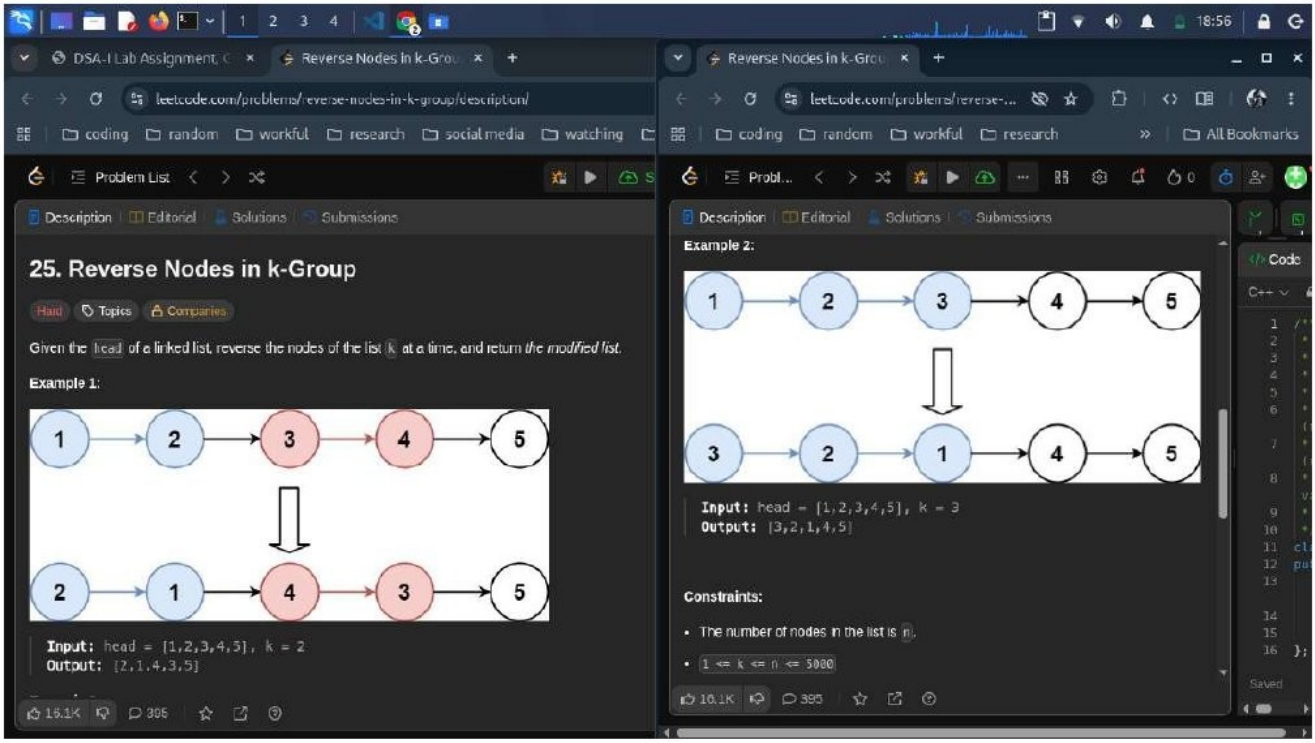
Question 3: Longest Increasing Path in a Matrix.....10
LeetCode 329 (Hard) – https://leetcode.com/problems/longest-increasing-path-in-a-matrix/

Question 4: Largest Rectangle in Histogram.....14
LeetCode 84 (Hard) – https://leetcode.com/problems/largest-rectangle-in-histogram/


DSA-I Lab Assignment
Trapping Rain Water - Le
leetcode.com/problems/trapping-rain-water/description/
Problem List
Submit
Premium
Description
Editorial
Solutions
Submissions
42. Trapping Rain Water
Attempted
Hard
Topics
Companies
Given n non-negative integers representing an elevation map where the width of each bar is 1, compute how much water it can trap after raining.
Example 1:
Input: height = [0,1,0,2,1,0,1,3,2,1,2,1]
Output: 6
Explanation: The above elevation map (black section) is represented by array [0,1,0,2,1,0,1,3,2,1,2,1]. In this case, 6 units of rain water (blue section) are being trapped.
37.3K
644
Code
C++

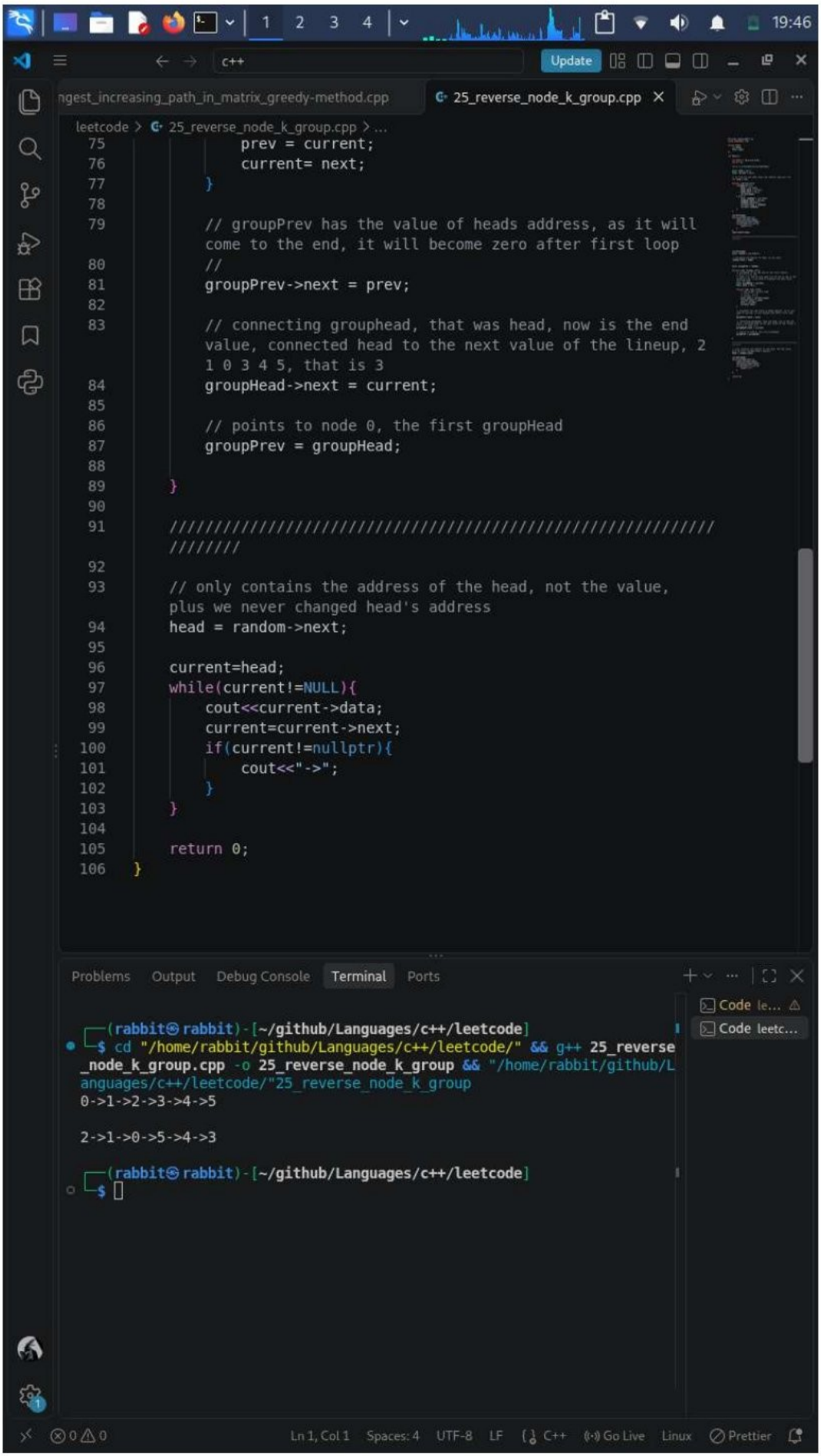

19:43
c++
Update
t_increasing_path_in_matrix_greedy-method.cpp
11_container_with_most_water.cpp
leetcode > 11_container_with_most_water.cpp > main()
#include <bits/stdc++.h>
using namespace std;
int main() {
// The height of each vertical bar
int arr[] = {1,8,6,2,5,4,8,3,7};
// Total number of bars (columns)
int col = sizeof(arr) / sizeof(arr[0]);
// Find the tallest bar to determine the number of rows
int row = arr[0];
for (int i = 0; i < col; i++) {
// If we find a taller bar, update the maximum height
if (row < arr[i]) {
row = arr[i];
}
}
// Create a 2D map to represent all the bars
// Each column represents one bar
// Each row represents one unit of height
int map[row][col];
// Fill the 2D map with 1s and 0s
for (int i = 0; i < col; i++) {
for (int j = 0; j < row; j++) {
// If the current row is below the bar's height,
// mark this position as part of the bar
if (j < arr[i]) {
map[j][i] = 1;
}
// Otherwise, this position is empty space
else {
map[j][i] = 0;
}
}
}
// Print the map from top row to bottom row
// This makes the bars look vertical when displayed
for (int i = row - 1; i >= 0; i--) {
for (int j = 0; j < col; j++) {
cout << map[i][j] << " ";
}
cout << endl;
}
//////////////////////////
// Store the maximum container area found so far
int biggest = 0;
// Choose the first bar of the container
Ln 40, Col 16
Spaces: 4
UTF-8
LF
C++
Go Live
Linux
Prettier

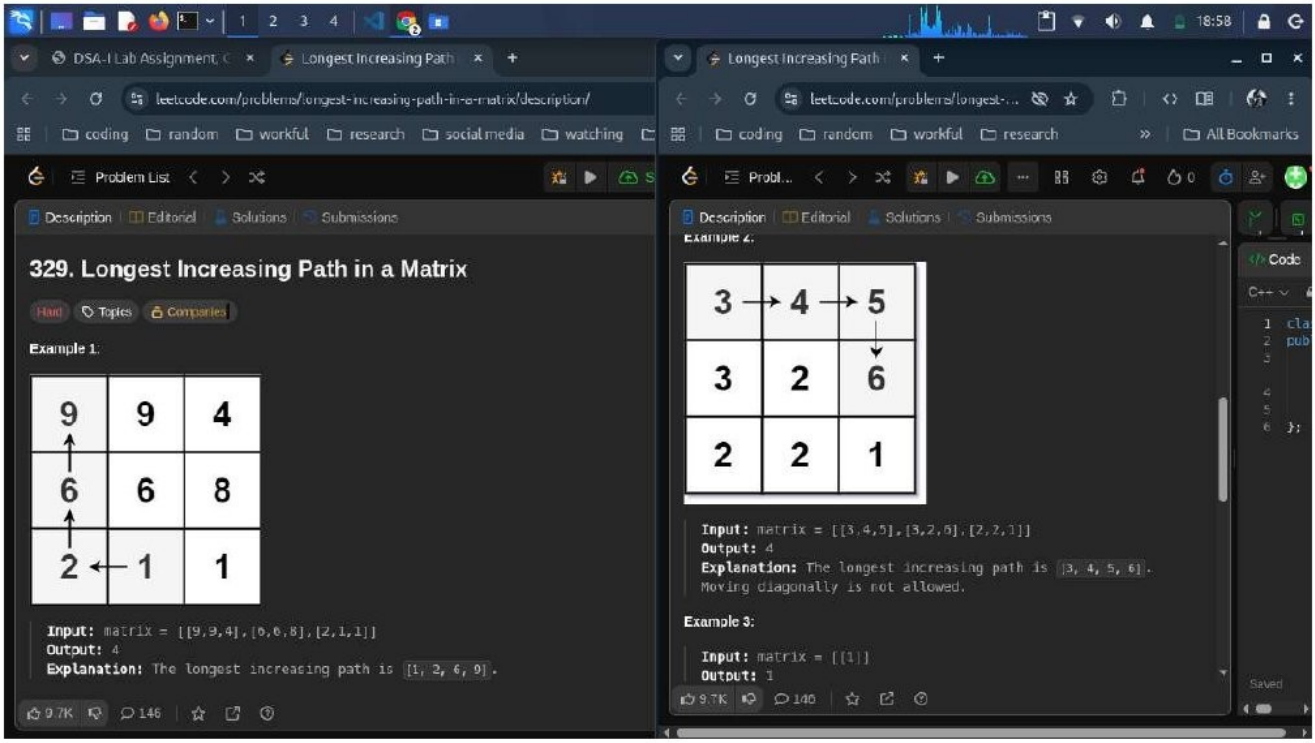

19:44
c++
Update
_increasing_path_in_matrix_greedy-method.cpp
11_container_with_most_water.cpp
leetcode > 11_container_with_most_water.cpp > main()
int main() {
//////////////////////////
// Store the maximum container area found so far
int biggest = 0;
// Choose the first bar of the container
for (int i = 0; i < col; i++) {
// Choose the second bar
// It must be to the right of the first bar
for (int j = i + 1; j < col; j++) {
// Water cannot be taller than the shorter bar
int height = min(arr[i], arr[j]);
// Distance between the two bars
int width = j - i;
// Calculate the water area between these two bars
int area = height * width;
// If this container holds more water than
// our previous best, update the maximum
if (area > biggest) {
biggest = area;
}
}
}
// Print the largest container area
cout << "\nMaximum water area: " << biggest;
return 0;
}
Problems
Output
Debug Console
Terminal
Ports
(rabbit㉿rabbit)-[~/github/Languages/c++/leetcode]
$ cd "/home/rabbit/github/Languages/c++/leetcode/" && g++ 11_container_with_most_water.cpp -o 11_container_with_most_water && "/home/rabbit/github/Languages/c++/leetcode/"11_container_with_most_water
0 1 0 0 0 0 1 0 0
0 1 0 0 0 0 1 0 1
0 1 1 0 0 0 1 0 1
0 1 1 0 1 0 1 0 1
0 1 1 0 1 1 1 0 1
0 1 1 0 1 1 1 1 1
0 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1
Maximum water area: 49
(rabbit㉿rabbit)-[~/github/Languages/c++/leetcode]
$
Code le...
Code leetc...
Ln 88, Col 39
Spaces: 4
UTF-8
LF
C++
Go Live
Linux
Prettier

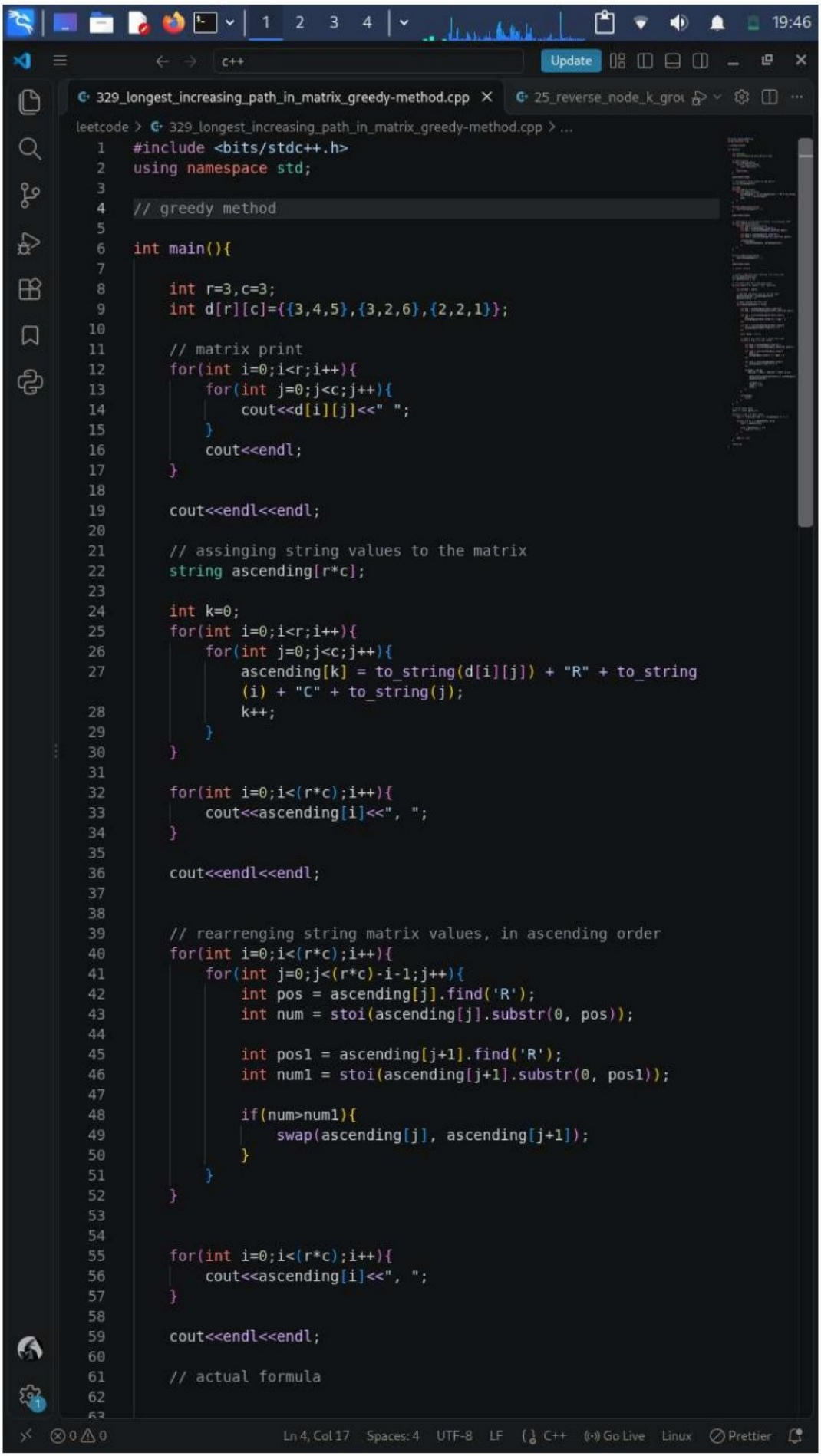

Reverse Nodes in k-Group
leetcode.com/problems/reverse-nodes-in-k-group/description/
Problem List
Description
Editorial
Solutions
Submissions
25. Reverse Nodes in k-Group
Hard
Topics
Companies
Given the head of a linked list, reverse the nodes of the list k at a time, and return the modified list.
Example 1:
Input: head = [1,2,3,4,5], k = 2
Output: [2,1,4,3,5]
Example 2:
Input: head = [1,2,3,4,5], k = 3
Output: [3,2,1,4,5]
Constraints:
The number of nodes in the list is n.
1 <= k <= n <= 5000
Code
C++


1 2 3 4
19:45
c++
Update
ngest_increasing_path_in_matrix_greedy-method.cpp
25_reverse_node_k_group.cpp
leetcode > 25_reverse_node_k_group.cpp > ...
#include <bits/stdc++.h>
using namespace std;
struct Node{
int data;
Node* next;
};
int main(){
int arr[] = {0,1,2,3,4,5};
int k = 3;
int n = sizeof(arr)/sizeof(arr[0]);
Node* head = NULL;
Node* current = NULL;
// to find out how many times the reverse loop will run
int loop = n/k;
for(int i=0;i<n;i++){
if(head==NULL){
head = new Node;
head->data = arr[i];
head->next = NULL;
current=head;
} else {
Node* newNode = new Node;
newNode->data = arr[i];
newNode->next=NULL;
current->next = newNode;
current= newNode;
}
}
current=head;
while(current!=NULL){
cout<<current->data;
current=current->next;
if(current!=nullptr){
cout<<"->";
}
}
cout<<endl<<endl;
//
////////
current=head;
Node* random = new Node();
// assinging the address of head, to use later
random->next = head;
Node* groupPrev = random;
for(int l=0; l<loop; l++){
Ln 1, Col 1
Spaces: 4
UTF-8
LF
C++
Go Live
Linux
Prettier

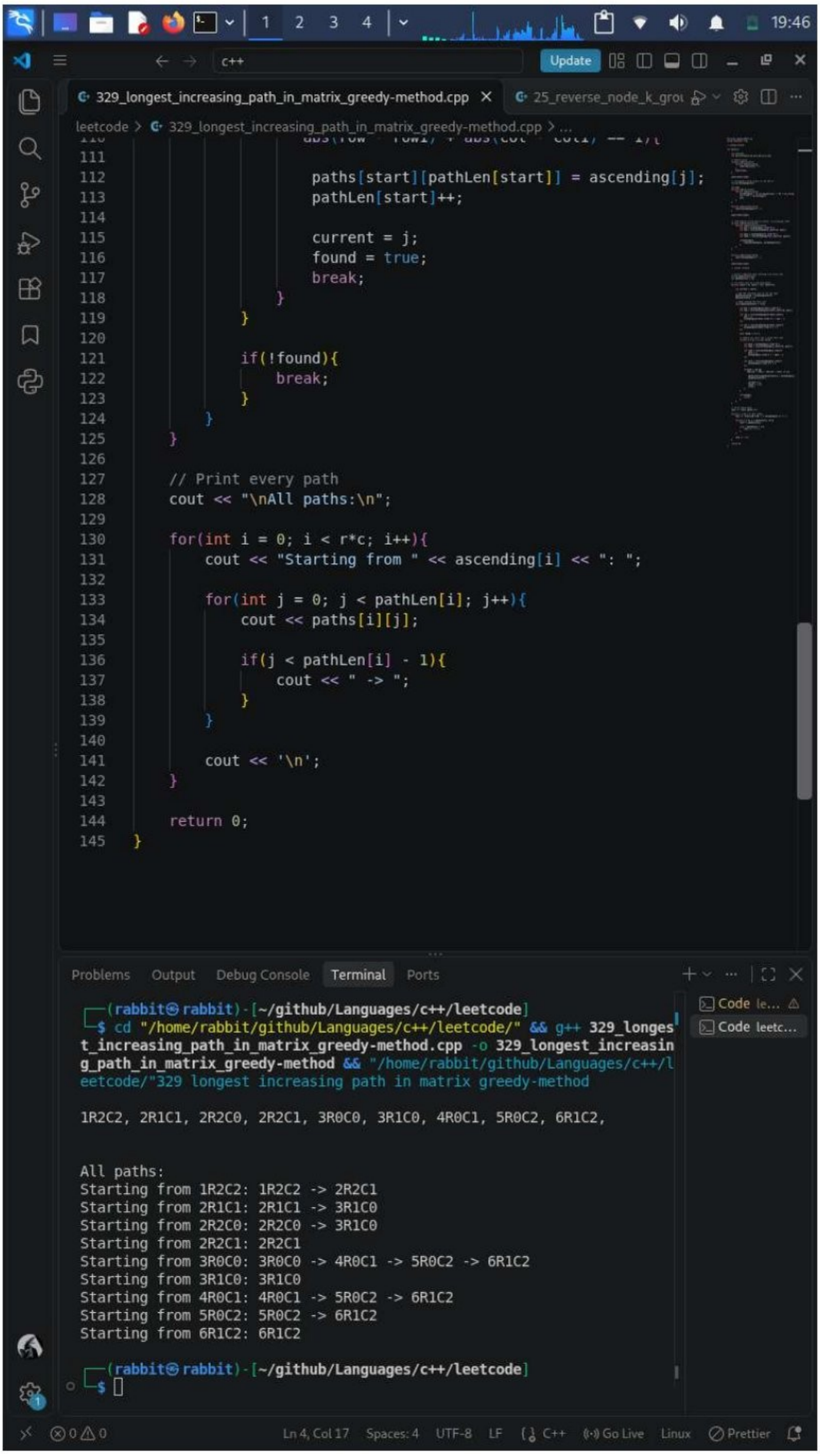

19:45
c++
Update
ngest_increasing_path_in_matrix_greedy-method.cpp
25_reverse_node_k_group.cpp
leetcode > 25_reverse_node_k_group.cpp > ...
current=head;
Node* random = new Node();
// assinging the address of head, to use later
random->next = head;
Node* groupPrev = random;
for(int l=0; l<loop; l++){
// grouphead is now the end of the first reverse
// 0 1 2 3 4 5, k=6
// head is 0, but we know head will be the at end in the upcoming loop, so we need to cuonnect the next value with the head
Node* groupHead = current;
Node* prev = NULL;
for(int i=0; i<k; i++){
// the normal reverse algo
// first : 2 1 0
// second: 5 4 3
Node* next = current->next;
current->next = prev;
prev = current;
current= next;
}
// groupPrev has the value of heads address, as it will come to the end, it will become zero after first loop
//
groupPrev->next = prev;
// connecting grouphead, that was head, now is the end value, connected head to the next value of the lineup, 2 1 0 3 4 5, that is 3
groupHead->next = current;
// points to node 0, the first groupHead
groupPrev = groupHead;
}
//
////////
// only contains the address of the head, not the value, plus we never changed head's address
head = random->next;
current=head;
while(current!=NULL){
cout<<current->data;
current=current->next;
if(current!=nullptr){
cout<<"->";
}
}
return 0;
}
Ln 1, Col 1
Spaces: 4
UTF-8
LF
C++
Go Live
Linux
Prettier

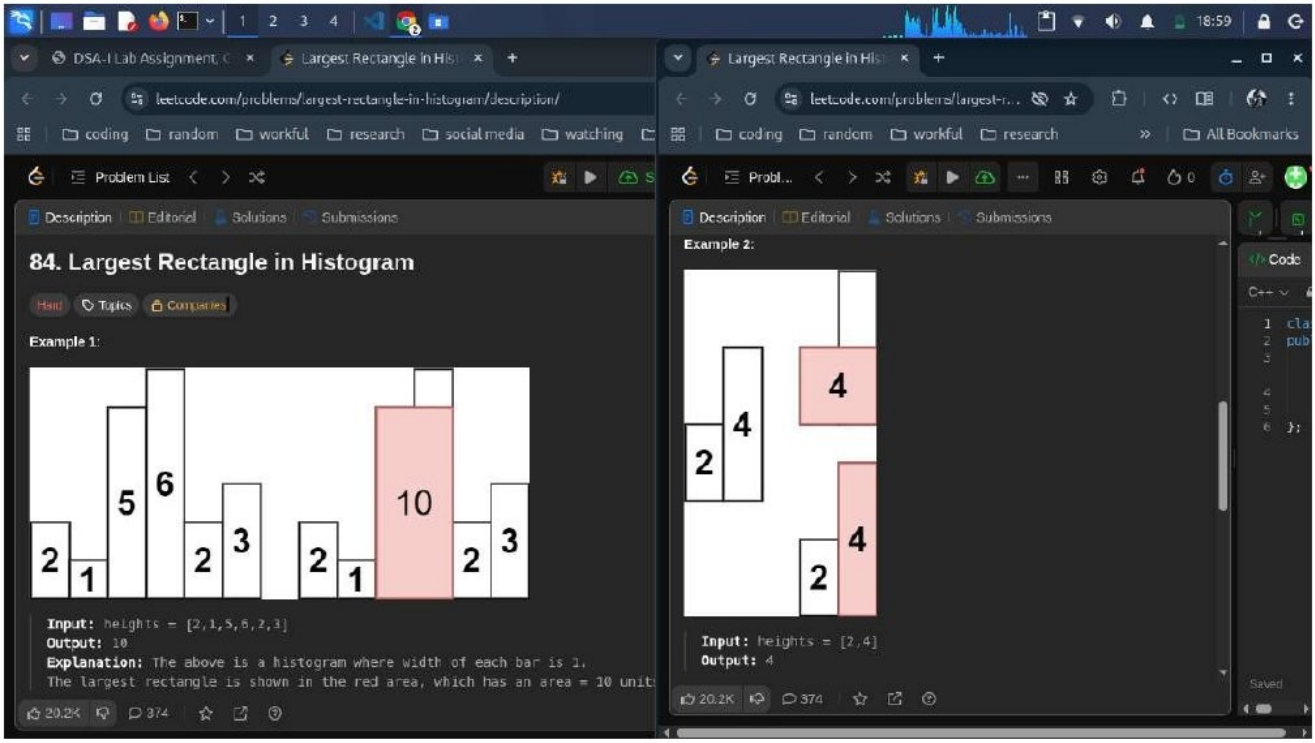

19:46
c++
Update
ngest_increasing_path_in_matrix_greedy-method.cpp
25_reverse_node_k_group.cpp
leetcode > 25_reverse_node_k_group.cpp > ...
prev = current;
current= next;
}
// groupPrev has the value of heads address, as it will come to the end, it will become zero after first loop
//
groupPrev->next = prev;
// connecting grouphead, that was head, now is the end value, connected head to the next value of the lineup, 2 1 0 3 4 5, that is 3
groupHead->next = current;
// points to node 0, the first groupHead
groupPrev = groupHead;
}
//
////////
// only contains the address of the head, not the value, plus we never changed head's address
head = random->next;
current=head;
while(current!=NULL){
cout<<current->data;
current=current->next;
if(current!=nullptr){
cout<<"->";
}
}
return 0;
}
Problems
Output
Debug Console
Terminal
Ports
Code le...
Code leetc...
(rabbit㉿rabbit)-[~/github/Languages/c++/leetcode]
$ cd "/home/rabbit/github/Languages/c++/leetcode/" && g++ 25_reverse_node_k_group.cpp -o 25_reverse_node_k_group && "/home/rabbit/github/Languages/c++/leetcode/"25_reverse_node_k_group
0->1->2->3->4->5
2->1->0->5->4->3
(rabbit㉿rabbit)-[~/github/Languages/c++/leetcode]
$
Ln 1, Col 1
Spaces: 4
UTF-8
LF
C++
Go Live
Linux
Prettier


Longest Increasing Path
leetcode.com/problems/longest-increasing-path-in-a-matrix/description/
Problem List
Description
Editorial
Solutions
Submissions
329. Longest Increasing Path in a Matrix
Hard
Topics
Companies
Example 1:
9 9 4
6 6 8
2 1 1
Input: matrix = [[9,9,4],[6,6,8],[2,1,1]]
Output: 4
Explanation: The longest increasing path is [1, 2, 6, 9].
9.7K
146
Example 2:
3 4 5
3 2 6
2 2 1
Input: matrix = [[3,4,5],[3,2,6],[2,2,1]]
Output: 4
Explanation: The longest increasing path is [3, 4, 5, 6].
Moving diagonally is not allowed.
Example 3:
Input: matrix = [[1]]
Output: 1
Code
C++


329_longest_increasing_path_in_matrix_greedy-method.cpp
leetcode > 329_longest_increasing_path_in_matrix_greedy-method.cpp > ...
#include <bits/stdc++.h>
using namespace std;
// greedy method
int main(){
int r=3,c=3;
int d[r][c]={{3,4,5},{3,2,6},{2,2,1}};
// matrix print
for(int i=0;i<r;i++){
for(int j=0;j<c;j++){
cout<<d[i][j]<<" ";
}
cout<<endl;
}
cout<<endl<<endl;
// assinging string values to the matrix
string ascending[r*c];
int k=0;
for(int i=0;i<r;i++){
for(int j=0;j<c;j++){
ascending[k] = to_string(d[i][j]) + "R" + to_string
(i) + "C" + to_string(j);
k++;
}
}
for(int i=0;i<(r*c);i++){
cout<<ascending[i]<<", ";
}
cout<<endl<<endl;
// rearrenging string matrix values, in ascending order
for(int i=0;i<(r*c);i++){
for(int j=0;j<(r*c)-i-1;j++){
int pos = ascending[j].find('R');
int num = stoi(ascending[j].substr(0, pos));
int pos1 = ascending[j+1].find('R');
int num1 = stoi(ascending[j+1].substr(0, pos1));
if(num>num1){
swap(ascending[j], ascending[j+1]);
}
}
}
for(int i=0;i<(r*c);i++){
cout<<ascending[i]<<", ";
}
cout<<endl<<endl;
// actual formula
Ln 4, Col 17
Spaces: 4
UTF-8
LF
C++
Go Live
Linux
Prettier


329_longest_increasing_path_in_matrix_greedy-method.cpp
leetcode > 329_longest_increasing_path_in_matrix_greedy-method.cpp > ...
// actual formula
// Build a separate path starting from every cell
string paths[r*c][r*c];
int pathLen[r*c] = {};
// Try every cell as a starting point
for(int start = 0; start < r*c; start++){
int current = start;
// Add the starting cell to its own path
paths[start][0] = ascending[start];
pathLen[start] = 1;
// Keep finding the next cell
while(pathLen[start] < r*c){
int pos = ascending[current].find('R');
int num = stoi(ascending[current].substr(0, pos));
int row = stoi(ascending[current].substr(
pos + 1,
ascending[current].find('C') - pos - 1
));
int col = stoi(ascending[current].substr(
ascending[current].find('C') + 1
));
bool found = false;
// Search all cells for a valid next cell
for(int j = 0; j < r*c; j++){
int pos1 = ascending[j].find('R');
int num1 = stoi(ascending[j].substr(0, pos1));
int row1 = stoi(ascending[j].substr(
pos1 + 1,
ascending[j].find('C') - pos1 - 1
));
int col1 = stoi(ascending[j].substr(
ascending[j].find('C') + 1
));
if(num1 > num &&
abs(row - row1) + abs(col - col1) == 1){
paths[start][pathLen[start]] = ascending[j];
pathLen[start]++;
current = j;
found = true;
break;
}
}
if(!found){
break;
}


329_longest_increasing_path_in_matrix_greedy-method.cpp
leetcode > 329_longest_increasing_path_in_matrix_greedy-method.cpp > ...
paths[start][pathLen[start]] = ascending[j];
pathLen[start]++;
current = j;
found = true;
break;
if(!found){
break;
// Print every path
cout << "\nAll paths:\n";
for(int i = 0; i < r*c; i++){
cout << "Starting from " << ascending[i] << ": ";
for(int j = 0; j < pathLen[i]; j++){
cout << paths[i][j];
if(j < pathLen[i] - 1){
cout << " -> ";
cout << '\n';
return 0;
Problems
Output
Debug Console
Terminal
Ports
(rabbit㉿rabbit)-[~/github/Languages/c++/leetcode]
$ cd "/home/rabbit/github/Languages/c++/leetcode/" && g++ 329_longest_increasing_path_in_matrix_greedy-method.cpp -o 329_longest_increasing_path_in_matrix_greedy-method && "/home/rabbit/github/Languages/c++/leetcode/"329 longest increasing path in matrix greedy-method
1R2C2, 2R1C1, 2R2C0, 2R2C1, 3R0C0, 3R1C0, 4R0C1, 5R0C2, 6R1C2,
All paths:
Starting from 1R2C2: 1R2C2 -> 2R2C1
Starting from 2R1C1: 2R1C1 -> 3R1C0
Starting from 2R2C0: 2R2C0 -> 3R1C0
Starting from 2R2C1: 2R2C1
Starting from 3R0C0: 3R0C0 -> 4R0C1 -> 5R0C2 -> 6R1C2
Starting from 3R1C0: 3R1C0
Starting from 4R0C1: 4R0C1 -> 5R0C2 -> 6R1C2
Starting from 5R0C2: 5R0C2 -> 6R1C2
Starting from 6R1C2: 6R1C2
(rabbit㉿rabbit)-[~/github/Languages/c++/leetcode]
$
Ln 4, Col 17
Spaces: 4
UTF-8
LF
C++
Go Live
Linux
Prettier


DSA-I Lab Assignment
Largest Rectangle in His
leetcode.com/problems/largest-rectangle-in-histogram/description/
coding
random
workful
research
social media
watching
Problem List
Description
Editorial
Solutions
Submissions
84. Largest Rectangle in Histogram
Hard
Topics
Companies
Example 1:
2
1
5
6
2
3
2
1
10
2
3
Input: heights = [2,1,5,6,2,3]
Output: 10
Explanation: The above is a histogram where width of each bar is 1.
The largest rectangle is shown in the red area, which has an area = 10 unit
20.2K
374
All Bookmarks
Example 2:
4
2
4
2
4
Input: heights = [2,4]
Output: 4
Code
C++
Saved


1 2 3 4
19:47
c++
Update
cpp
84_largest_rectangle_in_histogram.cpp
25_reverse_node_k_group.cpp
leetcode > 84_largest_rectangle_in_histogram.cpp > ...
#include <iostream>
using namespace std;
int main() {
int heights[] = {2,2, 1, 5, 6, 2, 3};
int size = sizeof(heights) / sizeof(heights[0]);
int biggest = 0;
for (int i = 0; i < size; i++) {
// pillars to multiply with
int pillars = 1;
// Expanding to left
for (int j = i - 1; j >= 0; j--) {
if (heights[j] >= heights[i]) {
pillars++;
} else {
break;
}
}
// Expanding to right
for (int j = i + 1; j < size; j++) {
if (heights[j] >= heights[i]) {
pillars++;
} else {
break;
}
}
int area = heights[i] * pillars;
if (area > biggest) {
biggest = area;
}
}
cout << biggest<<endl;
return 0;
}
Problems
Output
Debug Console
Terminal
Ports
Code le...
Code leetc...
(rabbit㉿rabbit)-[~/github/Languages/c++/leetcode]
$ cd "/home/rabbit/github/Languages/c++/leetcode/" && g++ 84_largest_rectangle_in_histogram.cpp -o 84_largest_rectangle_in_histogram && "/home/rabbit/github/Languages/c++/leetcode/"84_largest_rectangle_in_histogram
10
(rabbit㉿rabbit)-[~/github/Languages/c++/leetcode]
$
Ln 1, Col 1
Spaces: 4
UTF-8
LF
C++
Go Live
Linux
Prettier

JUSTIFICATION FILE (CO, PO and KP)

No.	Course Outcome (CO)	PO	KP
CO1	Demonstrate knowledge of fundamental data structures and algorithms.	PO(a) Engineering Knowledge	K2 – Conceptual-Based Mathematics K3 – Engineering Fundamentals K4 – Specialist Knowledge
CO2	Implement fundamental data structures and algorithms using programming tools to solve specified computational problems.	PO(e) Modern Tool Usage	K6 – Engineering Practice
CO3	Analyze problem requirements and justify the selection of appropriate data structures and algorithms.	PO(b) Problem Analysis	K3 – Engineering Fundamentals K4 – Specialist Knowledge
CO4	Design and develop algorithmic solutions for real-world problems using appropriate fundamental data structures and algorithms.	PO(c) Design/Development of Solutions	K5 – Engineering Design CP: P1, P2, P3

Term	What it means
CO – Course Outcome	A statement of what a student must be able to do by the end of the course. The four COs of this lab course are assessed through lab tests, quizzes, reports and implementation.
PO – Program Outcome	A broad ability that every graduate of the BSc in CSE program should have. Each CO supports one PO. PO(a) – Engineering Knowledge: applying mathematics and computing fundamentals to solve a problem. PO(b) – Problem Analysis: identifying and analysing a problem to reach well-supported conclusions. PO(c) – Design/Development of Solutions: designing a solution that meets the stated needs. PO(e) – Modern Tool Usage: using suitable modern tools to solve a problem.
KP – Knowledge Profile	The kind of knowledge needed to solve the problem. K2 – Conceptual-Based Mathematics: the mathematical ideas behind the subject. K3 – Engineering Fundamentals: the core ideas of computing used in the solution. K4 – Specialist Knowledge: detailed knowledge of the specific field. K5 – Engineering Design: knowledge used to design a complete solution. K6 – Engineering Practice: knowledge used to build, test and apply the solution in practice.
CP – Complex Problem	P1 – Depth of knowledge required; P2 – Range of conflicting requirements; P3 – Depth of analysis required. These are the attributes the course outline maps to CO4.

Justification of CO1

Relation to the assignment

CO1 asks for knowledge of the fundamental data structures and algorithms. This lab assignment covers them through four problems. Questions 1, 3 and 4 (pages 3 to 5, 10 to 13 and 14 to 15) use arrays, and Questions 1 and 3 also use a two-dimensional array (a map of bars and a matrix of cells). Question 2 (pages 6 to 9) uses a singly linked list built from a Node structure with a data field and a next pointer.

The algorithms are also fundamental: scanning every pair of bars with nested loops (Question 1), reversing a linked list in place with the prev, current and next pointers (Question 2), sorting by repeated comparison and swapping followed by a greedy walk to the next larger cell (Question 3), and expanding to the left and right of every bar (Question 4). Each problem starts with the LeetCode statement and examples, and the code that follows explains every step in comments, so the working of each structure and algorithm can be followed.

PO(a) – Engineering Knowledge

PO(a) expects mathematics and computing fundamentals to be applied to a problem. The programs apply the area formula height × width (Questions 1 and 4), the group count n / k to decide how many times the reversal loop runs (Question 2), and the condition |row − row1| + |col − col1| = 1 to find the four direct neighbours of a cell without moving diagonally (Question 3).

Knowledge Profile (KP)

K2 – Conceptual-Based Mathematics: the area formula, the neighbour condition and the group count. K3 – Engineering Fundamentals: arrays, matrices, structures, pointers and dynamic memory (new) in C++. K4 – Specialist Knowledge: group-wise linked-list reversal and the sorted-cell path method for the matrix.

Justification of CO2

Relation to the assignment

CO2 asks for the data structures and algorithms to be implemented with programming tools. All four problems are implemented in C++, written in VS Code, and compiled and run with g++ in the Linux terminal, as the screenshots show. LeetCode was used to read each problem statement and its examples.

The outputs show that the programs run and give the expected results. Question 1 prints the map of bars and the computed area (page 5). Question 2 prints the list 0->1->2->3->4->5 and then the reversed groups 2->1->0->5->4->3 for k = 3 (page 9). Question 3 prints the cells in ascending order and the path that starts from every cell (page 13). Question 4 prints 10, the same answer as the LeetCode example (page 15).

PO(e) – Modern Tool Usage

PO(e) expects modern tools to be used to solve a problem. The assignment uses an online judge platform (LeetCode) for the problem statements, the VS Code editor for writing the code, the g++ compiler for building it and the terminal for running it and checking the output.

Knowledge Profile (KP)

K6 – Engineering Practice: the solutions are written, compiled, run and checked as working programs. Comments in the code and printed intermediate results (the bar map, the list before and after reversal, the ordered cells and all paths) are used to verify each step.

Justification of CO3

Relation to the assignment

CO3 asks for the requirements of a problem to be analyzed and the choice of data structure and algorithm to be justified. For each question the input, the output and the rule of the problem were identified first, and a structure was then chosen to match. Arrays suit the bar heights of Questions 1 and 4 because the position of a bar is its index. A linked list is the natural structure for Question 2 because the problem is defined on nodes, and a dummy node keeps the new head when the first group is reversed. In Question 3 each cell is stored as a string made of its value, row and column, so that its position is not lost when the cells are sorted.

The table below shows the efficiency of the chosen methods. Here n is the number of elements, h is the height of the tallest bar, and N is rows × columns of the matrix.

Question	Data structure	Method used	Time	Space
Question 1	Array and a 2D map	Compare every pair of bars	$O(n^2 + n \cdot h)$	$O(n \cdot h)$
Question 2	Singly linked list	Reverse each group of k nodes in place	$O(n)$	$O(1)$ extra
Question 3	Matrix and string array	Sort the cells, then walk to a larger neighbour from every cell	$O(N^3)$	$O(N^2)$
Question 4	Array	Expand left and right from every bar	$O(n^2)$	$O(1)$

These methods were chosen because every step can be traced and printed, which suits a lab assignment. For larger inputs more efficient methods exist: a two-pointer scan for Question 1 (O(n) time), a memoised depth-first search for Question 3 (O(rows × columns) time) and a stack for Question 4 (O(n) time). Question 2 is already linear.

PO(b) – Problem Analysis

PO(b) expects a problem to be identified and analyzed to reach supported conclusions. Each question is analyzed by its requirement: only direct neighbours count in Question 3, and the width of a rectangle in Question 4 is the number of consecutive bars that are at least as tall as the chosen bar. The time and space of each method are then worked out and compared with better alternatives.

Knowledge Profile (KP)

K3 – Engineering Fundamentals: Big-O analysis of loops and memory. K4 – Specialist Knowledge: knowing when a linked list, a matrix or an array of encoded cells is the right structure.

Justification of CO4

Relation to the assignment

CO4 asks for algorithmic solutions to be designed and developed for real-world problems. Each question is developed from problem to working program: the steps are written as comments, test data is prepared (a list of heights, the list 0 to 5 with k = 3, and a 3 × 3 matrix), and intermediate results are printed so that the design can be checked. The problems model real situations: water held by a terrain profile (Question 1), reversing blocks of records in a sequence (Question 2), finding increasing routes across a grid of heights (Question 3) and the largest rectangular space under a skyline of bars (Question 4).

The designs handle the special cases of each problem. Only complete groups of k nodes are reversed because the loop runs n / k times (Question 2), the walk stops when no larger neighbour exists (Question 3), and the expansion stops at the first shorter bar on each side (Question 4).

PO(c) – Design/Development of Solutions

PO(c) expects a solution to be designed to meet stated needs. The programs take the sample input of each LeetCode problem, apply the designed steps and print the result the problem asks for.

Knowledge Profile (KP)

K5 – Engineering Design: the data structure is chosen first, the algorithm is designed in steps (for example the dummy node in Question 2 and the value-row-column code in Question 3), and the result is tested on sample data.

Complex Problem Attributes (P1, P2, P3)

P1: pointers, matrices and loop analysis are needed together, mainly in Questions 2 and 3. P2: simple, traceable code is balanced against time and space, as in Questions 3 and 4. P3: the complexity of each method is worked out before the alternatives are compared (CO3 page).